

Your Semester Role — Frontend Lead

Primary responsibility: Build and maintain the complete user-facing application for the Vulnerability Scanner.

By the end of the semester, you should be able to personally demonstrate:

Login → Dashboard → Configure Scan → Start Scan → Monitor Scan → View Vulnerabilities → Inspect CVE → View History → Risk Summary → Export/Remediation

Your existing prototype already establishes much of this flow: login, dashboard, recent scans, active scan status, severity totals, alerts, and a New Scan workflow.

Sprint 1 — Sept. 3–17

Frontend Foundation

Your objective

Turn the CPSC 490 prototype into the skeleton of the real application.

Your implementation work

Build the frontend project structure and routing/navigation for:

- Login
- Dashboard
- New Scan
- Scan Progress
- Scan Results
- Scan History
- Reports
- Settings

Create reusable components such as:

- Navigation/sidebar
- Header

- Buttons
- Form inputs
- Cards
- Tables
- Severity badges
- Loading indicators
- Error messages

Implement the basic **dark security-dashboard design** from CPSC 490.

Your original UX design specifically calls for a dark, professional interface emphasizing vulnerability severity, CVE identifiers, affected hosts, scan status, and recommended actions.

Functional work

Build a mocked New Scan interface containing:

- Target IP/hostname
- Port range
- Scan type
- Scan depth/intensity
- Start Scan button

Initially, the button can work against mock data.

What you should be able to demo

"This is the actual frontend application. I can navigate between all major views, configure a scan, and demonstrate the planned user workflow using mocked results."

Your Sprint 1 evidence

Code: frontend initialization + components + routes

Artifact: frontend implementation plan/component architecture

Tests: basic component/rendering tests

ChatGPT: evidence of evaluating frontend architecture, component organization, state-management choices, accessibility, etc.

Sprint 2 — Sept. 24–Oct. 1

Make the Frontend Functional

Now move away from mockups.

Your main objective:

Connect your frontend to the team's real backend.

Build the real New Scan workflow

Implement:

- Target IP/hostname field
- Port range
- Scan type
- Scan depth
- Input validation
- Submit/start scan
- API request
- Loading state
- API success handling
- API error handling

For example:

Invalid target

→ show validation error.

Backend unavailable

→ show connection error.

Scan accepted

→ navigate to scan progress/results.

The project requirements already define configuration validation, default parameters, invalid host handling, invalid port ranges, and unsupported configurations, so your UI should visibly support those flows.

Build an API/service layer

Don't put API calls directly throughout components.

You should own a frontend service structure conceptually like:

ScanService

AuthService

ResultsService

HistoryService

That gives you independently meaningful software architecture work, rather than simply HTML/CSS.

What you demonstrate

Configure Scan → Submit → Backend receives request → Frontend receives response

Your evidence

PRs should clearly show **your account** implementing the frontend/API integration.

Sprint 3 — Oct. 8–15

Results Dashboard + Midterm MVP

This is probably your **most important sprint** because the Midterm Demo is October 22.

Your frontend should become genuinely usable here.

Build Scan Results

Create the vulnerability results table.

Columns could include:

Severity	CVE	Host	Port	Service	CVSS
----------	-----	------	------	---------	------

Implement:

- Severity badges
- Sorting
- Filtering
- CVE details
- CVSS score
- Vulnerability description
- Remediation information

Your requirements explicitly call for results to be categorized by severity and presented in a structured dashboard where the user can filter/sort findings.

Build Dashboard Summary

Display:

- Total vulnerabilities
- Critical
- High
- Medium
- Low
- Recent scans
- Latest scan
- Overall risk

Build Scan History

Implement:

- Previous scan list
- Date/time
- Target
- Result summary
- Click to open previous result
- Empty history state
- Error state

Midterm acceptance criteria

By October 15, **your portion** should support:

**Login → Dashboard → New Scan → Configure → Start → Results → CVE
Details → Scan History**

Even if some data still comes from incomplete backend features, the UI workflow itself should work.

Midterm Demo — Oct. 22

You personally demonstrate the frontend.

Do not just stand there while somebody else clicks through it.

Your segment should be something like:

"I am responsible for the client-side application. I implemented the frontend architecture, reusable components, scan configuration workflow, API integration, dashboard, vulnerability results, and scan history."

Then actually demonstrate it.

That's particularly important because the syllabus requires each student to speak and show their own contribution during group presentations.

Sprint 4 — Oct. 29–Nov. 5

Advanced Frontend Features

Now turn the MVP into a more complete security application.

1. Real-Time Scan Progress

Build:

- Progress bar
- Percentage complete
- Current port/service
- Elapsed time
- Vulnerabilities found so far
- Running/completed/failed status
- Cancel/Pause controls if backend supports them

The CPSC 490 requirements specifically envision the dashboard receiving incremental progress information without a page refresh and then transitioning into the completed-results view.

2. Risk Assessment UI

Build a risk-summary section showing:

- Overall risk
- Severity distribution
- Critical findings
- Highest-risk hosts
- Recommendations

3. Vulnerability Detail View

Clicking a vulnerability should reveal progressively more information:

Level 1: CVE + severity

Level 2: CVSS + description + affected service

Level 3: remediation + references

That directly implements the progressive-disclosure approach already described in your UX design.

4. Remediation UI

Allow the user to assign:

- Open
- In Progress
- Resolved
- Accepted Risk

Plus remediation notes.

Sprint 5 — Nov. 12–19

Frontend Quality & Usability

Don't spend this sprint building ten more pages.

Your job changes from:

"Can I build it?"

to:

"Can users reliably use it?"

Error states

Test every major frontend failure:

- Invalid IP
- Invalid port range
- Backend unavailable
- Scan fails
- CVE API unavailable
- Empty results
- No vulnerabilities
- Large result set
- Authentication failure
- Scan still running
- Lost real-time connection

Loading states

Every network action should visibly communicate what is happening.

No button should simply appear frozen.

Empty states

For example:

No scan history

No previous scans have been performed.

No vulnerabilities

Scan completed. No known vulnerabilities were detected.

Usability testing

Have several people attempt tasks without you helping them:

Start a scan.

Find the highest-risk vulnerability.

Find CVSS information.

Find a previous scan.

Determine what remediation is recommended.

Record:

- Completion success
- Confusion
- Errors
- Time
- User feedback

Then modify the interface based on evidence.

That gives you excellent individual material for both the sprint artifact and final report.

Sprint 6 — Dec. 3–10

Frontend Release Candidate

No major new UI features.

Your responsibility becomes final product quality.

Polish

Fix:

- Layout inconsistencies
- Broken navigation
- Error handling
- Missing labels
- Overflow
- Large tables

- Loading behavior
- Button states
- Form validation
- Accessibility problems

Frontend testing

Complete:

- Component tests
- Form validation tests
- API error tests
- Navigation tests
- Important workflow/integration tests

Documentation

You should take primary responsibility for the **user-facing portions of the user manual** because you built the interaction model.

Document:

- 1. Login**
- 2. Dashboard**
- 3. Starting a Scan**
- 4. Configuring Scan Parameters**
- 5. Monitoring a Scan**
- 6. Reading Scan Results**
- 7. Understanding Severity/CVSS**
- 8. Viewing Previous Scans**
- 9. Exporting Reports**
- 10. Tracking Remediation**

Include screenshots.

The syllabus explicitly identifies technical documents such as user manuals and installation guides as part of the capstone's expected work.

Your semester checklist

By December, **you personally should have built or substantially owned:**

- Frontend application architecture
- Navigation/sidebar
- Login UI
- Dashboard
- New Scan interface
- Scan parameter validation
- Frontend API/service layer
- Scan progress interface
- Results table
- Severity visualization
- CVE detail view
- Filtering/sorting
- Scan history
- Risk summary
- Remediation interface
- Export/report interface
- Loading states
- Empty states
- Error handling
- Frontend tests
- Usability testing
- User documentation
- Final UI polish

That's a substantial individual contribution and gives you plenty to demonstrate independently.

What your six sprints boil down to

Sprint	Your Main Goal	Your Demonstrable Result
1	Frontend foundation	Navigable real application shell
2	Scan configuration/API	UI can initiate a real scan
3	Dashboard/results/history	Complete frontend MVP

Midterm	Demonstrate	Personally demonstrate frontend
4	Advanced UI	Live progress + risk/remediation
5	Quality	Tested and usability-validated UI
6	Release	Polished frontend + documentation
Final	Demonstrate	Complete end-to-end user workflow